

大学选课系统（5 个关系）建表：

- Students（学生）：StudentID（主码）、StudentName、Gender、Age、EnrollmentYear；性别只能为 '男' 或 '女'，年龄在 [15,100]，EnrollmentYear 为 4 位年份且不超过当前年份。
- Courses（课程）：CourseID（主码）、CourseName、Credits、DepartmentID（系别）、MaxCapacity；课程名不可重复，学分为正数，MaxCapacity 为正整数。
- Departments（系别）：DepartmentID（主码）、DepartmentName（唯一）。
- Enrollments（选课记录）：EnrollmentID（主码）、StudentID、CourseID、EnrollDate、Status（'选修'/'必修'/'退选' 等）；同一学生不能重复选同一门课（即 UNIQUE(StudentID, CourseID)）。
- Grades（成绩）：GradeID（主码）、EnrollmentID（外键）、Score（0-100，可为空表示未评分）、GradeDate（评分日期）；若有 Score，则 Score 必须在 0 到 100。

-- Departments 系别表

```
CREATE TABLE Departments (  
    DepartmentID    VARCHAR(20) PRIMARY KEY,  
    DepartmentName  VARCHAR(200) NOT NULL UNIQUE  
);
```

-- Students 学生表

```
CREATE TABLE Students (  
    StudentID      VARCHAR(20) PRIMARY KEY,  
    StudentName    VARCHAR(100) NOT NULL,  
    Gender         CHAR(1) NOT NULL,  
    Age            INTEGER NOT NULL CHECK (Age BETWEEN 15 AND 100),  
    EnrollmentYear INTEGER NOT NULL CHECK (EnrollmentYear >= 1900 AND EnrollmentYear <=  
EXTRACT(YEAR FROM CURRENT_DATE)),  
    CONSTRAINT chk_students_gender CHECK (Gender IN ('男', '女'))  
);
```

-- Courses 课程表

```
CREATE TABLE Courses (  
    CourseID       VARCHAR(20) PRIMARY KEY,  
    CourseName     VARCHAR(200) NOT NULL UNIQUE,  
    Credits        NUMERIC(3,1) NOT NULL CHECK (Credits > 0),  
    DepartmentID   VARCHAR(20) NOT NULL,  
    MaxCapacity    INTEGER NOT NULL CHECK (MaxCapacity > 0),  
    CONSTRAINT fk_courses_department FOREIGN KEY (DepartmentID)  
        REFERENCES Departments (DepartmentID)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

-- Enrollments 选课记录

```
CREATE TABLE Enrollments (  
    EnrollmentID   VARCHAR(30) PRIMARY KEY,  
    StudentID      VARCHAR(20) NOT NULL,  
    CourseID       VARCHAR(20) NOT NULL,  
    EnrollDate     DATE NOT NULL,
```

```

Status          VARCHAR(20) NOT NULL CHECK (Status IN ('选修','必修','退选')),
CONSTRAINT fk_enroll_student FOREIGN KEY (StudentID)
    REFERENCES Students (StudentID)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
CONSTRAINT fk_enroll_course FOREIGN KEY (CourseID)
    REFERENCES Courses (CourseID)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
-- 同一学生不能重复选同一门课
CONSTRAINT uq_enroll_student_course UNIQUE (StudentID, CourseID)
);

-- Grades 成绩表
CREATE TABLE Grades (
    GradeID        VARCHAR(30) PRIMARY KEY,
    EnrollmentID   VARCHAR(30) NOT NULL UNIQUE, -- 一个选课记录只对应一个成绩
    Score          NUMERIC(5,2) CHECK (Score >= 0 AND Score <= 100),
    GradeDate      DATE,
    CONSTRAINT fk_grade_enrollment FOREIGN KEY (EnrollmentID)
        REFERENCES Enrollments (EnrollmentID)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

```

一、基本语法骨架

```

CREATE TABLE 表名 (
    列名1 数据类型 [列级约束1] [列级约束2]...,
    列名2 数据类型 [列级约束]...,
    ...
    [表级约束1],
    [表级约束2]
);

```

- **核心:** 表名 + (列名 数据类型, ...)
- **约束:** 规则，用来保证数据正确性。分为**列级**（写在列定义后面）和**表级**（写在所有列定义完之后）。

二、常见数据类型

类型	用途	示例
INT / INTEGER	整数	Age INT
VARCHAR(n)	可变长度字符串（最常用），最多n个字符	StudentName VARCHAR(100)
CHAR(n)	固定长度字符串，总是占用n个字符空间	Gender CHAR(1)
NUMERIC(p,s) / DECIMAL(p,s)	精确小数。p=总位数, s=小数位数	Score NUMERIC(5,2) (总共5位, 2位小数, 如 100.00)
DATE	日期 (YYYY-MM-DD)	EnrollDate DATE
TIMESTAMP	日期+时间	LogTime TIMESTAMP

三、六大核心约束

约束类型	作用	列级写法 (推荐)	表级写法 (复合键/命名用)
主码 (PRIMARY KEY) 	唯一 + 非空 (实体完整性)	StudentID VARCHAR(20) PRIMARY KEY	CONSTRAINT pk_students PRIMARY KEY (StudentID)
外码 (FOREIGN KEY) 	引用另一张表的主码 (参照完整性)	(通常用表级写法)	CONSTRAINT fk_... FOREIGN KEY (列) REFERENCES 主表(主列)
唯一 (UNIQUE) 	值必须唯一，但允许有多个NULL	Email VARCHAR(100) UNIQUE	CONSTRAINT uq_... UNIQUE (Email)
非空 (NOT NULL) 	此列必须有值，不能是 NULL	StudentName VARCHAR(100) NOT NULL	(无表级写法)
检查 (CHECK) 	自定义规则，值必须满足括号内条件	Age INT CHECK (Age >= 18)	CONSTRAINT chk_... CHECK (Age >= 18)
默认 (DEFAULT) 	插入新行时不指定值，则使用默认值	Status VARCHAR(10) DEFAULT '正常'	(无表级写法)

四、复合主码/外码/唯一键 (表级约束的应用)

当多个列共同组成一个键时，必须使用表级约束。

- 复合主码:

```
CREATE TABLE Enrollments (
    StudentID VARCHAR(20),
    CourseID VARCHAR(20),
    ...
    PRIMARY KEY (StudentID, CourseID) -- 学生ID+课程ID 共同作为主码
);
```

- 复合唯一键:

```
-- 见上例, 如果主码不是(StudentID, CourseID), 但要保证其组合唯一
CONSTRAINT uq_student_course UNIQUE (StudentID, CourseID)
```

- 复合外码:

```
-- 假设主表的主码是复合的
FOREIGN KEY (Col1, Col2) REFERENCES OtherTable(PK1, PK2)
```

五、外码的级联操作 (ON UPDATE / ON DELETE) 💣

这是参照完整性的重点考点，定义了当主表的记录变化时，从表（有外码的表）该如何响应。

- FOREIGN KEY (列) REFERENCES 主表(主列) [ON UPDATE 操作] [ON DELETE 操作]

操作	效果	场景举例
RESTRICT	限制/禁止。如果从表有记录引用，则主表的操作失败。(默认)	想删除一个系列，但该系列下还有课程，操作失败。(最安全)
CASCADE	级联/跟随。主表删除/更新，从表所有引用记录也一起删除/更新。	删除一个学生的记录，他所有的选课记录也自动删除。(最常用)
SET NULL	置空。主表删除/更新，从表引用的外码列被设为 NULL。	一个课程被删除，选了这门课的学生的选课记录依然在，但课程ID变为空。(要求外码列允许为 NULL)

六、快速模板

1. 先写出 CREATE TABLE 表名 (...).
2. 依次定义每一列: 列名 数据类型。
3. 给每一列加上简单的列级约束 (PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT)。
4. 在所有列定义完后，集中处理表级约束，特别是复合键和所有外码。
5. 写外码时，别忘了 REFERENCES 谁，以及 ON DELETE / ON UPDATE 的级联策略。
6. CONSTRAINT 名字 是可选的，但写上更清晰，有助于排错。
7. 好的！我们用最精炼、最易懂的方式来搞定触发器 (Trigger)，让你在考场上胸有成竹。

触发器 (Trigger) 考场速成指南

任何一个触发器，都由这三部分组成，这也是考题的关键点：

1. 事件 (Event): 什么操作会“点燃”触发器？

- `INSERT`：当向表中插入新行时。
- `DELETE`：当从表中删除行时。
- `UPDATE`：当修改表中的行时。

2. 时机 (Timing): 事件发生前还是发生后执行？

- `BEFORE`：在事件发生之前触发。
 - 用途：
 - 数据校验：检查或修改即将插入/更新的数据。比如，插入的年龄如果是负数，可以在 `BEFORE INSERT` 里把它改成 0。
 - 取消操作：如果不满足某些条件，可以直接阻止这个事件的发生（通过引发错误）。
- `AFTER`：在事件发生之后触发。
 - 用途：
 - 日志记录：在数据被修改后，把操作记录到日志表中。
 - 级联操作：当一张表的数据变化后，去更新另一张相关联的表。比如，订单表插入一条记录后，去更新商品表的库存。

3. 动作 (Action): 触发后具体做什么？

- 这是一段 SQL 代码块（通常在 `BEGIN...END` 之间），里面可以包含 `INSERT`, `UPDATE`, `DELETE`, `SELECT` 等几乎所有 SQL 语句。

3. `NEW` 和 `OLD`

在触发器的动作代码里，你可以访问到两个特殊的“虚拟表”或“行变量”，它们包含了事件相关的数据：

- `NEW`：代表新数据。
 - 在 `INSERT` 事件中，`NEW` 包含了即将被插入的新行数据。
 - 在 `UPDATE` 事件中，`NEW` 包含了修改之后的新行数据。
 - 在 `DELETE` 事件中，`NEW` 不可用。
- `OLD`：代表旧数据。
 - 在 `DELETE` 事件中，`OLD` 包含了即将被删除的旧行数据。
 - 在 `UPDATE` 事件中，`OLD` 包含了修改之前的旧行数据。
 - 在 `INSERT` 事件中，`OLD` 不可用。

如何使用: `NEW.列名` 或 `OLD.列名`，如 `NEW.Score` 表示新插入或更新后的成绩。`OLD.Status` 表示更新或删除前的状态。

4. 触发器语法模板

```
CREATE TRIGGER 触发器名 -- 1. 给它起个名字
{BEFORE | AFTER} {INSERT | DELETE | UPDATE} -- 2. 确定“时机”和“事件”
ON 表名 -- 3. 绑定到哪张表上
FOR EACH ROW -- 4. 这是“行级触发器”，每影响一行就触发一次（考试基本都是这种）
BEGIN
    -- 5. 触发后要执行的“动作”代码
    -- 在这里可以使用 NEW 和 OLD
    -- 例如：IF NEW.Score < 0 THEN SET NEW.Score = 0; END IF;
END;
```

- **FOR EACH ROW**: 意味着如果一个 **UPDATE** 语句修改了 10 行，那么这个触发器就会执行 10 次，每次处理一行。

场景一：自动记录操作日志

- **需求**: 每当 **Students** 表有学生信息被删除时，需要将该学生的学号和姓名备份到 **Student_Log** 表中。
- **分析**:
 - **事件**: **DELETE**
 - **时机**: **AFTER** (因为要等删除成功后再记录)
 - **表**: **Students**
 - **动作**: 向 **Student_Log** 插入数据，数据来源是 **OLD** 行。
- **代码**:

```
CREATE TRIGGER trg_log_student_delete
AFTER DELETE ON Students
FOR EACH ROW
BEGIN
    INSERT INTO Student_Log (StudentID, StudentName, Action, LogTime)
    VALUES (OLD.StudentID, OLD.StudentName, 'DELETED', NOW());
END;
```

场景二：数据校验与自动修正

- **需求**: 当向 **Grades** 表插入或更新成绩时，如果 **Score** 低于 0，则自动修正为 0；如果高于 100，则自动修正为 100。
- **分析**:
 - **事件**: **INSERT, UPDATE**
 - **时机**: **BEFORE** (因为要在数据写入磁盘前修改它)
 - **表**: **Grades**
 - **动作**: 使用 **IF** 语句判断 **NEW.Score** 并进行修改。
- **代码**:

```
CREATE TRIGGER trg_check_score
BEFORE INSERT ON Grades -- 可以再写一个 BEFORE UPDATE 的, 或在某些数据库中写成 BEFORE
INSERT OR UPDATE
FOR EACH ROW
BEGIN
    IF NEW.Score < 0 THEN
        SET NEW.Score = 0;
    ELSEIF NEW.Score > 100 THEN
        SET NEW.Score = 100;
    END IF;
END;
```

场景三：级联更新，维护数据一致性

- 需求: 有一个 Courses 表和一个 Departments 表, Departments 表中有一个 CourseCount 字段, 记录该系别的课程总数。要求每当 Courses 表增加或删除课程时, 自动更新 Departments 表的 CourseCount。
- 分析 (以插入为例):
 - 事件: INSERT
 - 时机: AFTER (等课程确实插入成功了再计数)
 - 表: Courses
 - 动作: UPDATE Departments SET CourseCount = CourseCount + 1 WHERE DepartmentID = NEW.DepartmentID;
- 代码 (插入):

```
CREATE TRIGGER trg_update_course_count_add
AFTER INSERT ON Courses
FOR EACH ROW
BEGIN
    UPDATE Departments
    SET CourseCount = CourseCount + 1
    WHERE DepartmentID = NEW.DepartmentID;
END;
```

(删除课程的触发器与此类似, 只是把 + 1 改成 - 1, 并使用 OLD.DepartmentID)

NEW 和 OLD 变量在触发器中的含义

事件 (Event)	时机 (Timing)	OLD 变量	NEW 变量
INSERT	BEFORE INSERT	不可用 (不存在旧数据)	即将要插入的新行数据。 可以修改 NEW 中的值。
	AFTER INSERT	不可用 (不存在旧数据)	已经插入成功的新行数据。 NEW 的值只读。
UPDATE	BEFORE UPDATE	修改之前的旧行数据。 OLD 的值只读。	将要更新为的新行数据。 可以修改 NEW 中的值。
	AFTER UPDATE	修改之前的旧行数据。 OLD 的值只读。	已经更新完成的新行数据。 NEW 的值只读。
DELETE	BEFORE DELETE	将要被删除的旧行数据。 OLD 的值只读。	不可用 (不存在新数据)
	AFTER DELETE	已经被删除的旧行数据。 OLD 的值只读。	不可用 (不存在新数据)